

Tolerância a falhas

STD29006 – Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



(A) DELL R440



(B) DELL R340



Qual servidor poderá garantir uma disponibilidade maior?

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



Fonte: Wikimedia

- Fonte de energia redundante
hotswap aumenta a disponibilidade, é fácil de trocar e pode ser trocada com o servidor ligado

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



Fonte: Wikimedia

- Fonte de energia redundante
hotswap aumenta a disponibilidade, é fácil de trocar e pode ser trocada com o servidor ligado
- Se faltar energia no prédio, como resolver?

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



Fonte: Sociedade Esportiva Bandeirante, Brusque - SC

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



Fonte: Sociedade Esportiva Bandeirante, Brusque - SC



Se esse gerador a diesel falhar?

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



Fonte: Generac

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



Fonte: Generac

Como aumentar a disponibilidade?

Sistema computacional que executa exclusivamente em uma única máquina



Fonte: Generac

- Para garantir, ou melhor, para aumentar a disponibilidade, fazemos uso de redundância de componentes (i.e. fonte, grupo gerador, etc)
- Temos uma cadeia de dependência entre componentes
- No exemplo: sistema computacional → *hardware* → sistema de energia

Tolerância a falhas

Sistemas distribuídos

- Uma das características que difere os sistemas distribuídos daqueles que executam em uma única máquina é a noção de **falha parcial**
- Parte do sistema falha, enquanto as demais partes continuam funcionando e aparentemente de forma correta

Meta crucial para o projeto de sistemas distribuídos

Garantir que possa se recuperar automaticamente de falhas parciais sem afetar seu desempenho, ou seja, é esperado que seja um **sistema confiável** ou também chamado de **tolerante a falhas**

Sistemas confiáveis (*Dependable systems*)

Confiabilidade (*dependability*), segundo Tanenbaum e Steen (2008)

Um **componente** A depende de um **componente** B se a corretude do comportamento de A depender da corretude do comportamento de B

Sistemas confiáveis (*Dependable systems*)

- **Facilidade de manutenção** (*maintainability*)

- O quão fácil o sistema poderá ser recuperado

- **Segurança de funcionamento** (*safety*)

- Estar operacional ou não executar aquilo para o qual foi especificado sem que provocar danos a outros sistemas ou pessoas que dele dependam

- **Disponibilidade** (*availability*)

- Estar operacional para ser usado no instante que o usuário precisar (prontidão)

- **Confiabilidade** (*reliability*)

- Estar operacional e capaz de atender a especificação durante um intervalo de tempo e não apenas em um instante (continuidade)

Confiabilidade e Disponibilidade

- A **confiabilidade** $C(t)$ de um componente é a probabilidade condicional que este operou corretamente durante o intervalo $[0, t)$, dado que estava operando corretamente no início do período
- A **disponibilidade** $D(t)$ de um componente é a probabilidade de estar operacional em um instante determinado

Exemplo

Um avião deve ser **confiável durante todo voo** e depois pode passar um **longo período de manutenção**, que afeta sua disponibilidade mas não sua confiabilidade

Confiabilidade e Disponibilidade

Sistemas de missão críticos (Weber, 2002)

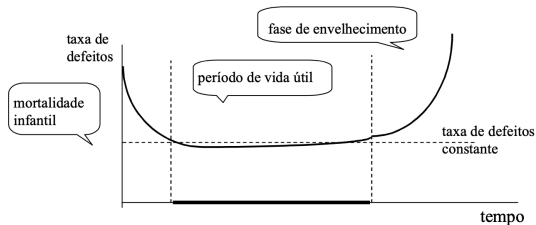
- Curtos períodos de operação incorreta são inaceitáveis
- Quando o reparo é impossível

Exemplos

- **Usinar nuclear, aeronaves, exploração espacial**
 - Exigem alta confiabilidade, mas podem aceitar baixa disponibilidade
- **Sistemas bancários, sites de comércio eletrônico**
 - Exigem alta confiabilidade e alta disponibilidade

Confiabilidade e Disponibilidade

Tempo de vida e taxa de defeito



Fonte: (Weber, 2002)

■ Componentes de hardware

- Possuem um tempo de vida útil
- Taxa de erros aumenta a medida que envelhecem

■ Componentes de software

- Taxa de defeito maior no início
- Após a fase de testes essa taxa tende a diminuir

Classificação de sistemas de acordo com a disponibilidade

classe	nível	indisponibilidade	
		anual	mensal
contínuo	99.9999%	31.5 s	2.59 s
tolerante a falhas	99.999%	5.26 m	25.9 s
alta disponibilidade	99.9%	8.76 h	43.8 m
disponibilidade normal	99%	3.65 d	7.20 h

$$Disponibilidade = \frac{tempodisponivel}{tempodisponivel + tempoindisponivel}$$



Disponibilidade 100% é inviável atingir, pois quanto mais próximo de 100%, mais custoso será

Gmail - Detalhes do serviço

[Painel de status do Google Workspace](#)

Esta página contém informações de desempenho dos Serviços do Google a seguir. A menos que o contrário seja indicado, essas informações de status são aplicáveis aos serviços para consumidores e aos serviços para organizações que utilizam o Google Workspace.

Volte a qualquer momento para ver o status atual dos serviços listados abaixo. Para saber mais ou relatar um problema, acesse a [Central de Ajuda do Google Workspace](#) ou a página [Problemas conhecidos do Google Workspace](#).

Horário	Descrição
 14/12/20 09:52	O problema com o Gmail foi resolvido para a maioria dos usuários afetados. Continuaremos o trabalho para restaurar o serviço dos outros usuários, mas publicaremos mais atualizações no Painel de status do Google Workspce. Tenha certeza de que a confiabilidade do sistema é uma prioridade para o Google, e estamos fazendo aprimoramentos contínuos para tornar os nossos sistemas melhores.
 14/12/20 09:47	O serviço do Gmail já foi restaurado para alguns usuários e esperamos uma resolução para todos os usuários em breve. Este período é apenas uma estimativa e pode sofrer alterações. Os usuários afetados não conseguem acessar o Gmail.
 14/12/20 09:31	O serviço do Gmail já foi restaurado para alguns usuários e esperamos uma resolução para todos os usuários em breve. Este período é apenas uma estimativa e pode sofrer alterações.
 14/12/20 08:55	Estamos cientes do problema com o Gmail que está afetando a maioria dos usuários. Os usuários afetados não conseguem acessar o Gmail. Forneceremos uma atualização às 14/12/20 09:12 com detalhes sobre quando o problema deve ser resolvido. Observe que esse tempo de resolução é uma estimativa e pode sofrer alterações.

Todos os horários são exibidos no fuso horário local, a menos que sejam definidos de outra forma.

 [Feed RSS](#)

 Nenhum problema  Interrupção do serviço  Falha temporária do serviço

Episódios de indisponibilidade e painéis de monitoramento

■ Painéis de monitoramento de serviços em nuvem

- <https://status.cloud.google.com/summary>

- <https://metastatus.com/>

- <https://health.aws.amazon.com/>

- <https://azure.status.microsoft/>

■ Episódios de indisponibilidade

- <https://status.fortilan.forticloud.com/incidents/svkh1n2zcd1z>

- <https://www.pingdom.com/outages/internet-outages-the-12-most-impactful>

Definições sobre defeito, erro e falha

- **Defeito** (*failure*)

- Desvio da especificação

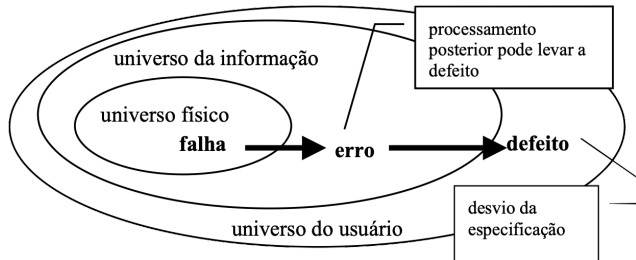
- **Erro** (*error*)

- Sistema em estado de **erro** (ou em um estado errôneo), se o processo posterior a partir desse estado puder levá-lo a um **defeito**

- **Falha** (*fault*)

- Causa física ou algorítmica do erro
- Ex: problemas na especificação, implementação, ou operação; componentes defeituosos; fadiga de componentes, interferências e variações ambientais

Definições sobre defeito, erro e falha

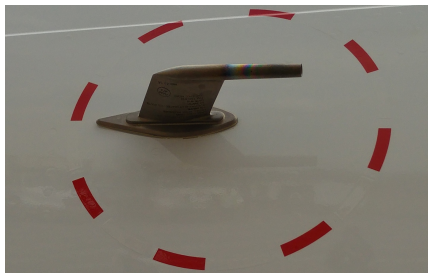


Fonte: (Weber, 2002)

Exemplificando defeito, erro e falha

Tubo de pitot

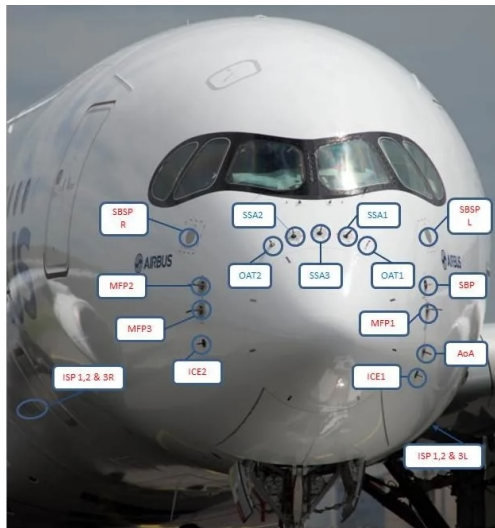
Instrumento de medição de velocidade, que na aviação é conectado a instrumentos como altímetro, indicador de velocidade no ar e indicador de velocidade vertical



Fonte: Wikipedia

- **Falha:** Congelamento do tubo pitot
- **Erro:** Avião a 400km/h, porém envia o valor de 840km/h
- **Defeito:** Piloto não consegue aumentar a velocidade

Curiosidade sobre instrumentos de medição



ISP = Integrated Static Port

SBSP = StandBy Static Port (ISIS)

MFP = Multi Functional Probe (Pitot, AoA & TAT)

AoA = Angle of Attack (Adiru#1 and PFCS)

SBP = StandBy Pitot (ISIS)

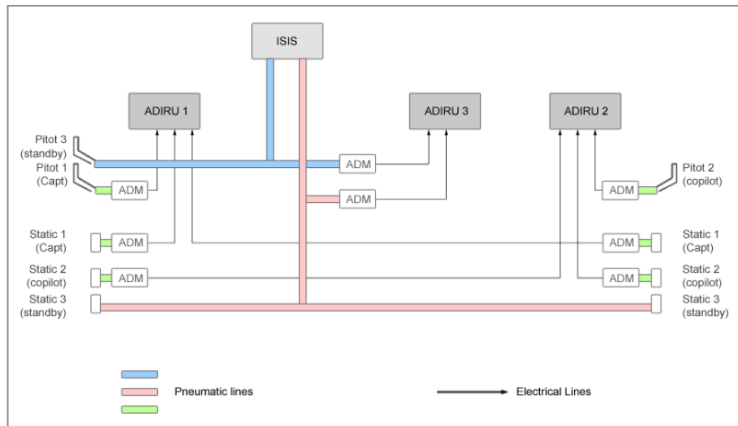
SSA = Side Slip Angle

OAT = Outside Air Temp.

ICE = Ice Detector

Airbus A350 - Fonte: <https://www.airliners.net/forum/viewtopic.php?t=1382051#p20047969>

Curiosidade sobre instrumentos de medição



Sistema de medição de velocidade de um A330 - Fonte:
<https://www.bea.aero/docspa/2009/f-cp090601.en/pdf/f-cp090601.en.pdf>

Como lidar com falhas

■ Prevenção

- Projeto que minimize sua ocorrência
- Validação formal, inspeção de código, teste e uso de hardware robusto

■ Remoção

- Uma vez encontradas, podem ser removidas por meio de testes, depuração, substituição de componentes

■ Tolerar

- Assuma que falhas (ou potenciais) sempre existirão
- Sistema deve continuar funcionando mesmo diante de falhas

Como lidar com falhas

■ Prevenção

- Projeto que minimize sua ocorrência
- Validação formal, inspeção de código, teste e uso de hardware robusto

■ Remoção

- Uma vez encontradas, podem ser removidas por meio de testes, depuração, substituição de componentes

■ Tolerar

- Assuma que falhas (ou potenciais) sempre existirão
- Sistema deve continuar funcionando mesmo diante de falhas



Para sistemas que exigem alta confiabilidade ou alta disponibilidade, em sua concepção deve-se fazer uso de técnicas de tolerância a falhas

Falhas

- **Permanente**

- se mantém ao longo do tempo

- **Intermitente**

- ocorre eventualmente devido a instabilidades de hardware ou software

- **Transitória**

- resultado de condições temporárias e que a princípio não se repetirá

Tipos de falhas

■ Parada

- Processo para de funcionar, mas estava funcionando corretamente até então

■ Omissão

- O processo falha ao receber ou enviar mensagens que são esperadas que ele receba ou envie

■ Temporização

- Responde após o intervalo de tempo limite

■ Arbitrária

- O processo continua a funcionar, porém produz uma saída incorreta a qual não é capaz de detectar que esteja errada
- Também chamada de falhas bizantinas

Tipos de falhas

■ Parada

- Processo para de funcionar, mas estava funcionando corretamente até então

■ Omissão

- O processo falha ao receber ou enviar mensagens que são esperadas que ele receba ou envie

■ Temporização

- Responde após o intervalo de tempo limite

■ Arbitrária

- O processo continua a funcionar, porém produz uma saída incorreta a qual não é capaz de detectar que esteja errada
- Também chamada de falhas bizantinas



Qual tipo de falha para o exemplo do tubo de *pitot* congelado?

Falha de parada

Cenário de exemplo

Componente A não está recebendo mais mensagens do componente B, pode-se concluir que B teve uma falha de parada?

- Na prática é impossível determinar se houve uma falha de parada ou de omissão
- Em **sistemas síncronos** o processamento e a entrega de mensagens possuem um tempo de expiração associado, permitindo detectar de maneira confiável uma falha por omissão ou de temporização
- Em **sistemas assíncronos** não é possível assumir tempos máximos para processamento ou envio de mensagens e assim, não é possível detectar de maneira confiável que aconteceu uma falha de parada

Mascarando falhas por meio da redundância

Redundância pode ser usada para mascarar ou tolerar falhas. Para mascarar são necessários mais componentes redundantes

- **Redundância de informação**

- Códigos de correção de erros (ECC, *error correction code*)

- **Redundância temporal**

- Repete a computação, adequada para falhas intermitentes ou transitórias

- **Redundância física** (*hardware* ou *software*)

- Várias réplicas de um equipamento
- Em *software* não é desejado usar cópias idênticas, mas sim verificar a consistência entre as réplicas



Toda forma de redundância impacta no sistema, seja no custo de implantação ou operação, desempenho ou tamanho

Tolerância a falhas em sistemas distribuídos

Como detectar um processo falho?

Detectores de falhas

- Módulos acoplados a cada processo para monitorar os demais processos
- Classificados como confiáveis ou não confiáveis, com base em suas características de completude e acurácia
- Podem ser implementados por meio de troca de mensagens de telemetria (*heartbeat*)

Detectores de falhas

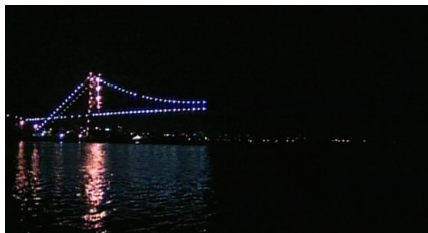
- **Detectores não confiáveis** indica que o processo é
 - **Não suspeito** – recebeu evidências recentes que o processo não falhou
 - Ex: mensagem foi recebida há pouco tempo
 - **Suspeito** – possui alguma indicação que o processo falhou recentemente
 - Ex: nenhuma mensagem recebida dentro do intervalo esperado
 - Um processo correto, porém com uma execução mais lenta pode ser considerado suspeito
- **Detectores confiáveis** indica que o processo é
 - **Não suspeito** – igual ao anterior
 - **Falho** – conseguiu determinar que o processo falhou

Detetores de falhas

- **Detetores não confiáveis** indica que o processo é
 - **Não suspeito** – recebeu evidências recentes que o processo não falhou
 - Ex: mensagem foi recebida há pouco tempo
 - **Suspeito** – possui alguma indicação que o processo falhou recentemente
 - Ex: nenhuma mensagem recebida dentro do intervalo esperado
 - Um processo correto, porém com uma execução mais lenta pode ser considerado suspeito
- **Detetores confiáveis** indica que o processo é
 - **Não suspeito** – igual ao anterior
 - **Falho** – conseguiu determinar que o processo falhou

Detetores confiáveis só podem ser implementados em sistemas síncronos, mas usando descobertas de vários **detetores não confiáveis** é possível que a suspeita seja aceita por todos os processos, em algum momento

Grupos e mascaramento de falhas



http://pt.wikipedia.org/wiki/Blecaute_na_Ilha_de_Santa_Catarina_em_2003

- **Agrupamento de processos idênticos** permite mascarar um ou mais processos falhos
- Duas pontes com o mesmo objetivo
- **Independência de falhas** é quando se busca evitar falhas que afetem todos os processos
- Parte da iluminação da ponte é de uma subestação no continente e parte é de outra na ilha

Grupos e mascaramento de falhas

- Um grupo é chamado de **tolerante a f falhas** se puder mascarar **f falhas** concorrentes de seus membros
 - Todos os processos devem ser idênticos
 - Todos os membros processam as mesmas mensagens e na mesma ordem

Total de processos para um grupo **tolerante a f falhas**?

- **Falhas de parada, omissão, temporização** : $n \geq f + 1$, pois nenhum membro produzirá saída incorreta, assim basta que uma réplica seja correta
- **Falhas arbitrárias** : pelo menos $n \geq 2f + 1$, pois o resultado correto só pode ser obtido com o voto da maioria

Consenso

- Com a replicação de processos é desejado que esses entrem em **acordo** quanto as ações que serão executadas por todos, mesmo na presença de falhas arbitrárias
- Todos processos corretos devem contar com os mesmos dados e sobre os quais deverão aplicar um mesmo algoritmo de decisão
- Todas as mensagens serão processadas na mesma ordem por todos os processos – **difusão seletiva com ordem total**

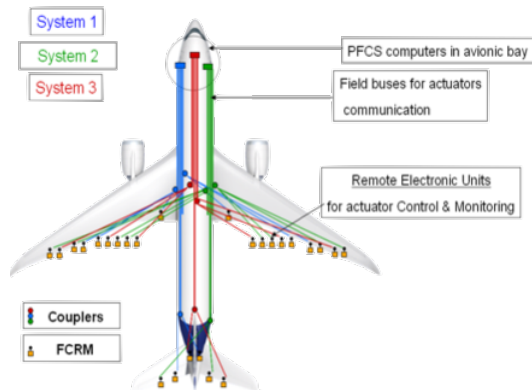
Problema dos Generais Bizantinos – Lamport, 1982

- Lamport demonstrou que é possível alcançar consenso sob falhas arbitrárias em sistemas síncronos
- Fisher¹ em 1985 provou que é impossível obter consenso distribuído em sistemas assíncronos considerando qualquer modelo de falha

¹ <https://www.the-paper-trail.org/post/2008-08-13-a-brief-tour-of-flp-impossibility>

Problema dos Generais Bizantinos – Lamport, 82

Surgiu da necessidade de consolidar sinais de sensores de altitude replicados em um avião



- Ter um único sensor é muito arriscado!
- Tendo mais de um sensor, como saber qual o valor correto, dado que alguns destes podem falhar?

Problema dos Generais Bizantinos – Lamport, 82

- Cenário: **N** generais cercam uma cidade inimiga e precisam decidir, em conjunto, atacar ou recuar
- Comunicação apenas por mensageiros, sem canal seguro
- Alguns generais podem ser traidores, enviando mensagens falsas para confundir os demais



Problema dos Generais Bizantinos – Lamport, 82

- **Objetivo:** garantir que todos os generais leais cheguem à mesma decisão (atacar ou recuar)
- Se o comandante for leal, todos os leais devem seguir sua ordem



Dificuldade do Consenso Bizantino

- Traidores podem enviar mensagens diferentes para cada general, criando confusão
- Exemplo: O comandante ordena atacar, mas um traidor diz a alguns para recuar
- Se não houver maioria leal, o grupo pode se dividir e falhar



Limite para Solução

Condição necessária

- Só é possível garantir consenso se mais de 2/3 dos generais forem leais
- Com apenas 3 generais, sendo 1 traidor, então não há solução confiável
- Para tolerar f traidores, são necessários pelo menos $n \geq 3f + 1$ generais

Problema dos Generais Bizantinos – Lamport, 82

Premissas

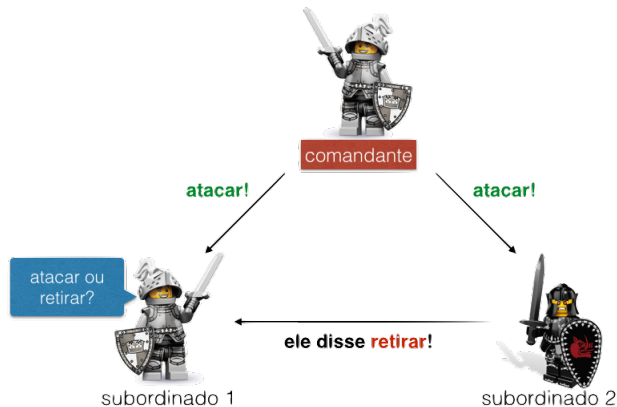
- **P1:** Toda mensagem enviada é entregue corretamente
 - **P2:** O receptor sabe quem enviou a mensagem
 - **P3:** A perda de mensagem é detectável
 - Se não receber mensagem, assume-se a ordem “retirar”
-
- **P1** e **P2** impedem que um traidor altere ou falsifique mensagens entre generais
 - **P3** impede que um traidor cause desacordo apenas deixando de enviar mensagens
 - Ou seja: sistema síncrono, comunicação ordenada, unicast e atraso limitado

Problema dos Generais Bizantinos – Lamport, 82

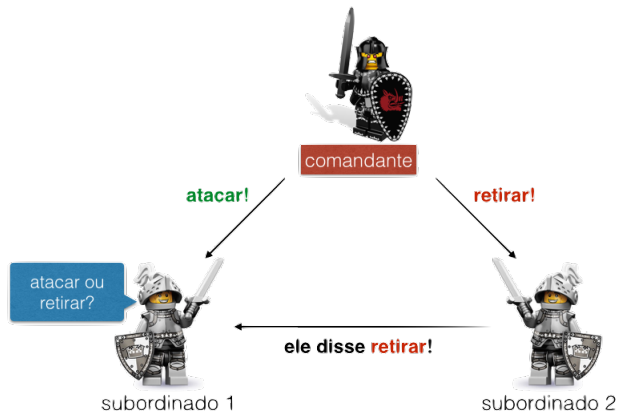
Algoritmo resumido

- Entre os n generais, um é o comandante e os demais são subordinados
- O comandante envia sua ordem a todos os subordinados
 - **C1:** Todos os subordinados leais devem decidir pela mesma ação
 - **C2:** Se o comandante for leal, todos os subordinados leais devem seguir sua ordem

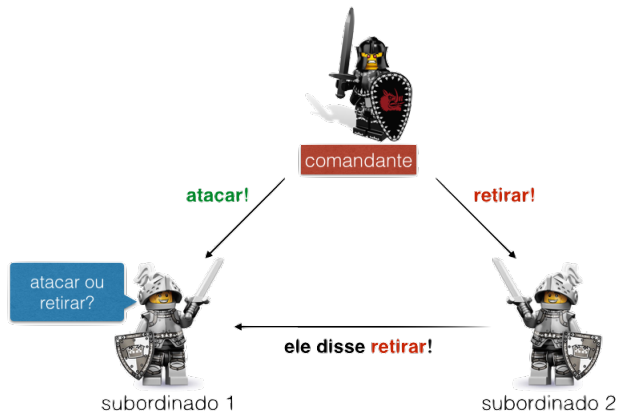
Problema dos Generais Bizantinos – Exemplos



Problema dos Generais Bizantinos – Exemplos



Problema dos Generais Bizantinos – Exemplos



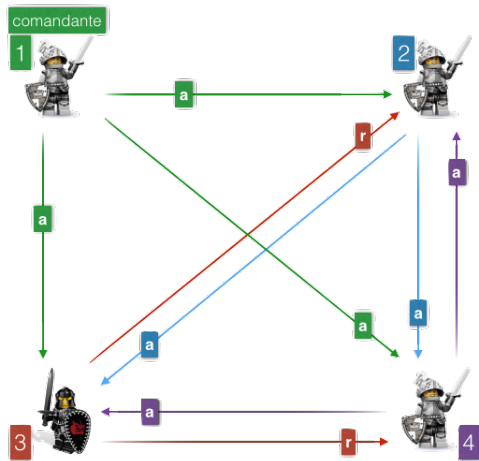
- Em ambos os casos, o subordinado 1 não consegue distinguir a ordem correta nem identificar o traidor

Solução para o Problema Bizantino

Condição: $n \geq 3f + 1$

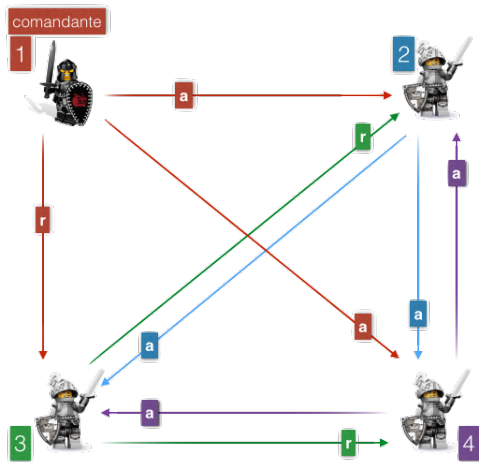
- Cada general constrói um vetor V de tamanho n
 - $V[i]$ armazena o valor recebido do general i
 - Se i for leal, $V[i]$ contém sua ordem; caso contrário, pode ser indefinido
- Cada general adota o valor da **maioria** em V como decisão final

Problema dos Generais Bizantinos – Lamport, 82



2	a	r	a	= a
3	TRAIOR			
4	a	a	r	= a

Problema dos Generais Bizantinos – Lamport, 82



2	a	r	a	= a
3	r	a	a	= a
4	r	a	a	= a

Blockchain, bitcoin e algoritmo de consenso

- Em 2008 Satoshi Nakamoto tornou publico seu artigo "*Bitcoin: a peer-to-peer electronic cash system*"²
- Resolveu o problema do "gasto duplo com ativos digitais"
- Algoritmo de consenso "prova de trabalho " (PoW, *Proof of Work*) permite aos nós que compõem a rede chegar a um acordo sobre o próximo bloco que será adicionado na cadeia de blocos
 - **Princípio:** É difícil encontrar a solução, porém é rápido e fácil verificá-la posteriormente

²<https://bitcoin.org/bitcoin.pdf>

Essa aula foi baseada em



COULORIS, George; DOLLIMORE, Jean; KINDBERG, Tim. **Sistemas Distribuídos: Conceitos e projeto**. 5. ed.: Bookman, 2013.

<https://app.minhabiblioteca.com.br/books/9788582600542/>.



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.



WEBER, Taisy Silva. **Um roteiro para exploração dos conceitos básicos de tolerância a falhas**. 2002. Disponível em:

<https://www.inf.ufrgs.br/~taisy/disciplinas/textos/index.html>. Acesso em: 1 out. 2023.